Chunking

Large Language Models (LLMs) process text in units called tokens. However, every model has a limit on the amount of information it can process in a single request. When working with large documents, providing the entire document may be inefficient or exceed the model's context limits.

Text chunking helps address this problem by dividing large documents into smaller, manageable sections. These sections can then be retrieved individually and provided to an LLM when relevant to a user's question. To understand why text chunking is important, we first need to understand tokens, context length, and context windows.

❖ Understanding Tokens and Context Length

1. Token

- A token is a small piece of text processed by an LLM. It can be a complete word, part of a word, a number, or punctuation.
- A tokenizer is the component that splits text into tokens before the model processes it.
- Example: “I love learning AI.”
- Illustrative tokens: `I` | `love` | `learning` | `AI` | `.`
- The actual number of tokens depends on the tokenizer. Tokens are important because LLMs measure text input and output limits in tokens.

2. Context Length

- **Context length** refers to the maximum number of tokens a model can process within its context limits for a request.
- The context may include the user's prompt, conversation history, retrieved information, and generated output, depending on the model and its architecture.
- Different models support different context lengths. A larger context length allows a model to process more information in a single request.

3. Context Window

- The **context window** is the model's available working context for processing information during a request. It can include input tokens and, subject to the model's limits, output tokens.

4. Why Is Context Important?

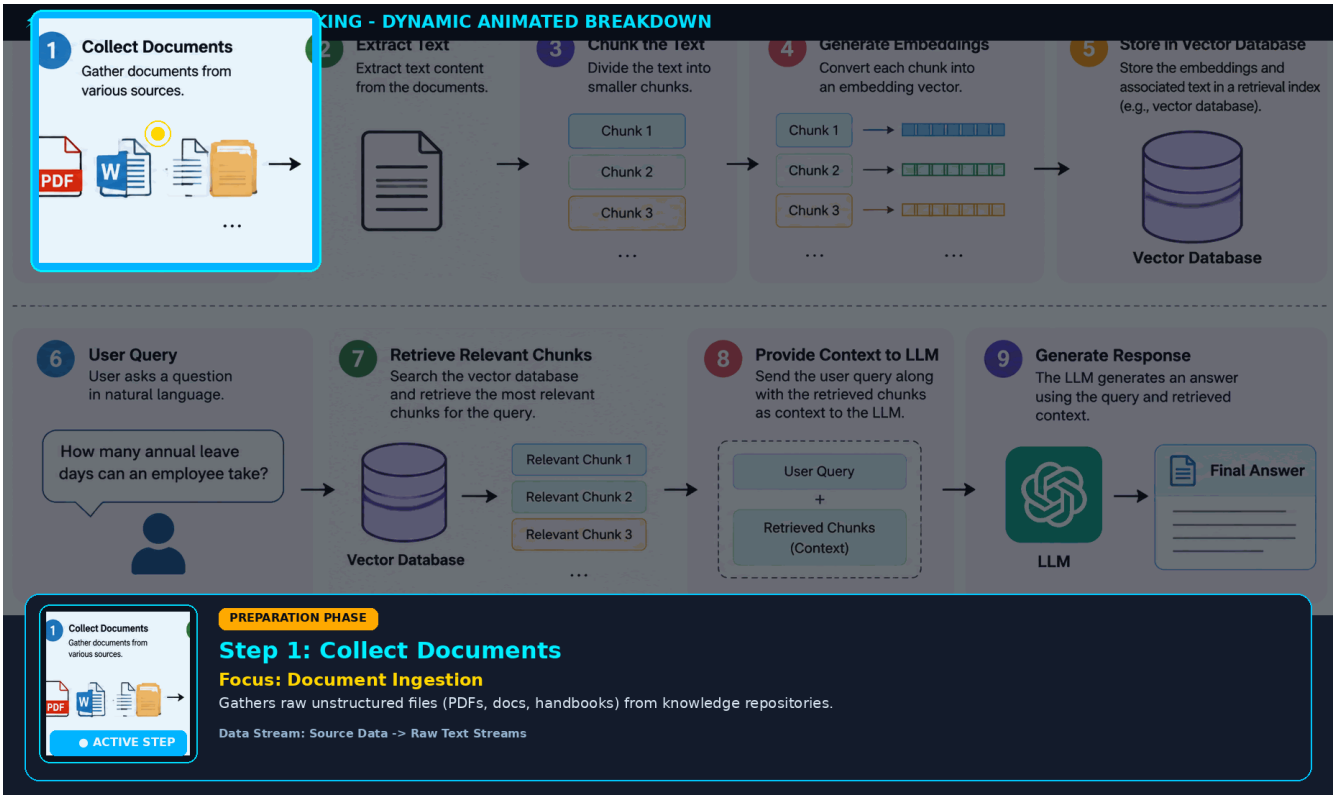
- Context helps an LLM understand the information surrounding a question and identify relationships between different parts of the input.
- **Longer context:** Can help the model connect information across longer documents.
- **Focused context:** Helps the model concentrate on information relevant to the question.
- **Unnecessary context:** May consume available tokens and increase processing time and cost.
- A larger context window does not automatically produce better answers. The relevance and organization of the information also matter.

❖ What Is Text Chunking?

- Text chunking is the process of dividing a large document or text into smaller sections called *chunks*.
- Instead of processing an entire document as one large piece, an application can divide it into chunks that are easier to retrieve and manage.
- **Example:** Imagine a 200-page employee handbook. If someone asks about annual leave, the system can retrieve the relevant annual leave section instead of providing the entire handbook.

❖ What Is Text Chunking in RAG?

- Retrieval-Augmented Generation (RAG) combines information retrieval with an LLM to generate answers grounded in relevant external information.
- In a typical RAG pipeline:



❖ Why Is Text Chunking Required?

Text chunking is useful for several reasons:

- Manage large documents: Breaks lengthy content into manageable sections.
- Improve retrieval: Helps retrieve information relevant to a user's question.
- Use context efficiently: Reduces the need to pass entire documents to the LLM.
- Preserve useful information: Appropriate chunk boundaries and overlap can retain important context.
- Reduce irrelevant content: Helps avoid including unrelated sections in the model's input.

Chunking does not automatically improve every response. Its effectiveness depends on the document structure, chunking method, chunk size, overlap, and retrieval approach.

❖ Key Parameters of Text Chunking

1. Chunk Size

Chunk size determines how much text each chunk contains. It may be measured in tokens, characters, sentences, or other units depending on the implementation.

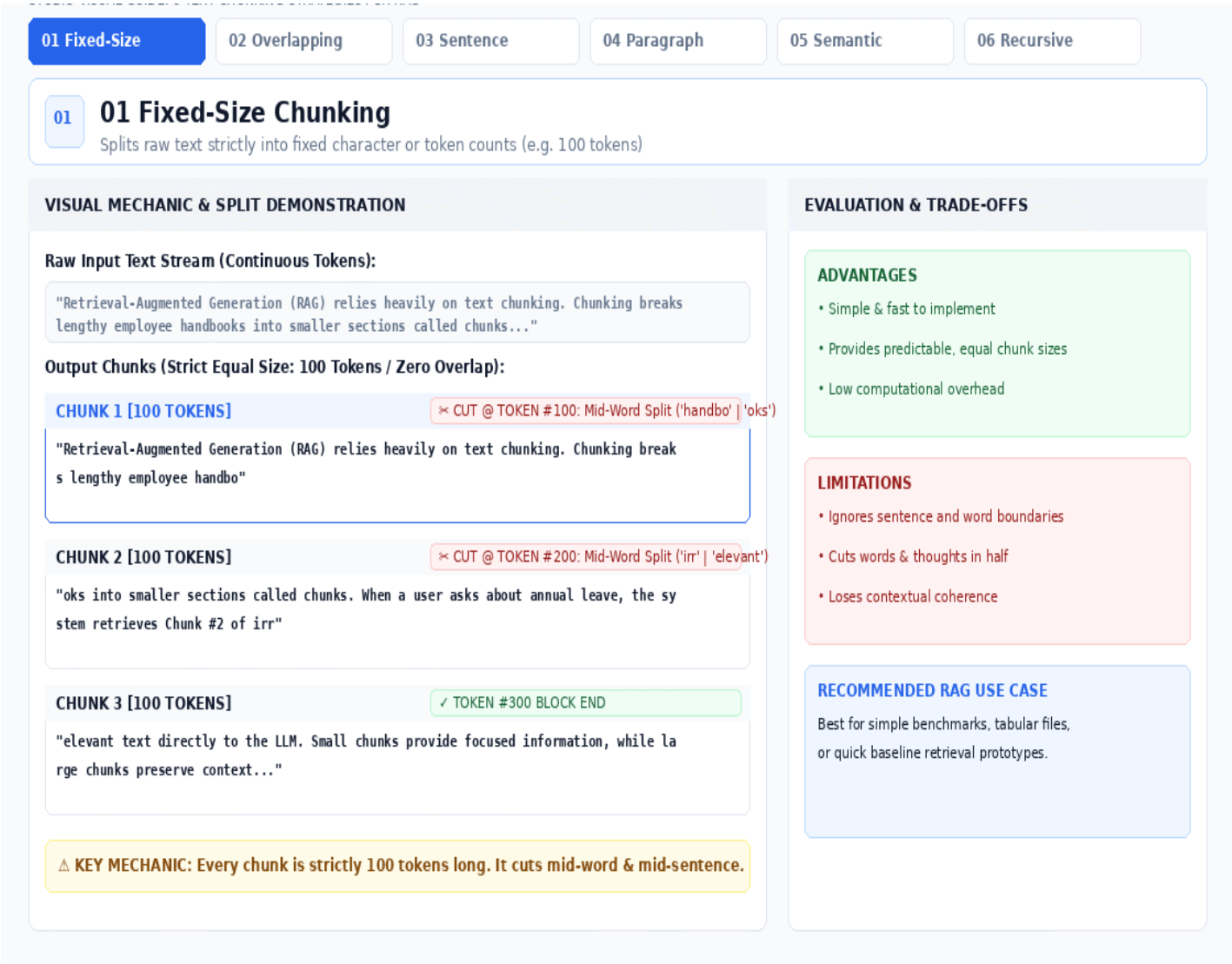
- Smaller chunks: Often provide more focused information but may lose surrounding context.
- Larger chunks: Preserve more surrounding information but may include irrelevant content.

2. Chunk Overlap

- Chunk overlap is the amount of text shared between consecutive chunks.
- For example, if a system creates chunks of 100 tokens with an overlap of 20 tokens, the next chunk reuses the previous chunk's final 20 tokens in a simple sliding-window approach.
- Overlap can help preserve context when an important sentence or idea crosses a chunk boundary. However, excessive overlap creates redundant text and increases storage and processing requirements.

❖ Types of Chunking Techniques

- Chunking techniques define how a large document is divided into smaller pieces. The choice of technique depends on the document structure, the type of information, and the application's retrieval requirements.



❖ Practical Implementation

A practical implementation of Text Chunking is available in the Jupyter Notebook linked below. The notebook demonstrates how large text is divided into smaller chunks using different chunking techniques and configurable chunk sizes and overlap. It also compares the outputs to understand how chunking affects context preservation and information retrieval.

You can access the project from this link: **Text Chunking – Jupyter Notebook**